

**FACULTY
OF MATHEMATICS
AND PHYSICS**
Charles University

BACHELOR THESIS

Adam Balko

Arduino platform simulator

Department of Software and Computer Science Education

Supervisor of the bachelor thesis: RNDr. David Bednárek, Ph.D.

Study programme: Computer Graphics, Vision and
Computer Games Development

Prague 2026

I declare that I carried out this bachelor thesis on my own, and only with the cited sources, literature and other professional sources. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In date

Author's signature

Dedication.

Title: Arduino platform simulator

Author: Adam Balko

Department: Department of Software and Computer Science Education

Supervisor: RNDr. David Bednárek, Ph.D., Department of Software Engineering

Abstract: Use the most precise, shortest sentences that state what problem the thesis addresses, how it is approached, pinpoint the exact result achieved, and describe the applications and significance of the results. Highlight anything novel that was discovered or improved by the thesis. Maximum length is 200 words, but try to fit into 120. Abstracts are often used for deciding if a reviewer will be suitable for the thesis; a well-written abstract thus increases the probability of getting a reviewer who will like the thesis.

Keywords: digital circuit simulator, Arduino

Název práce: Simulátor prostředí Arduino

Autor: Adam Balko

Katedra: Katedra softwaru a výuky informatiky

Vedoucí bakalářské práce: RNDr. David Bednárek, Ph.D., Katedra softwarového inženýrství

Abstrakt: Abstrakt práce přeložte také do češtiny.

Klíčová slova: simulátor číslicových obvodů, Arduino

Contents

Introduction	6
0.1 Comparison of existing solutions	6
0.2 Specific goals of the thesis	6
1 Architecture design	7
1.1 Architecture overview	7
1.2 Inter-process communication	8
1.2.1 Considerations	8
1.2.2 Snapshotting	9
1.2.3 Events	9
1.2.4 TCP stream	10
1.2.5 Shared memory	11
1.3 Backend	11
1.3.1 The simulation	11
1.3.2 Events and IPC	13
1.3.3 Utilities	13
1.4 Frontend	14
2 Implementation	15
2.1 Backend	15
2.1.1 Simulator module	15
2.1.2 Executable module	15
2.1.3 IPC Adapters	15
2.2 Frontend	15
2.2.1 Component Management project	15
2.2.2 Main project with Avalonia UI	15
2.2.3 IPC Adapter project	15
3 Developer documentation	16
3.1 Compiling and running Arduino code	16
3.2 Defining components	16
3.3 Defining scenes	16
3.4 IPC interface	16
Conclusion	17
List of Figures	18
List of Tables	19
List of Abbreviations	20
A Attachments	21
A.1 First Attachment	21

Introduction

Arduino is a very popular microcomputer among amateurs and professionals alike. Despite this, testing the code for an Arduino machine can be fairly difficult. Debugging options are limited and the user needs to rely on additional measuring equipment. The usual choice is to test the code with an AVR emulator. Emulation is the process of running machine code on a hardware that doesn't match its instruction set. AVR is the processor that controls the Arduino board. These emulators are sometimes part of a bigger logical circuit simulation system, other times they are just a single piece of software.

Goal of this work is to skip the emulation process and create a working Arduino simulator by mimicking the changes of digital values on the board's pinout. Bundled with this simulator is a graphic application for creating and testing logical circuits, with Arduino as it's main component. The point of this application is to attach to the running Arduino simulator and visualize the expected Arduino behavior with user's code on a given circuit.

0.1 Comparison of existing solutions

0.2 Specific goals of the thesis

1 Architecture design

1.1 Architecture overview

The whole project is divided into two main parts. The purpose of the backend is to execute the Arduino code and simulate the change on the board's pinout. The purpose of the frontend is to reflect these changes on a user defined logical circuit, while offering the user control over the simulation. We chose to implement the backend in C++ and the frontend in C#. Both backend and frontend are running in its own separate process.

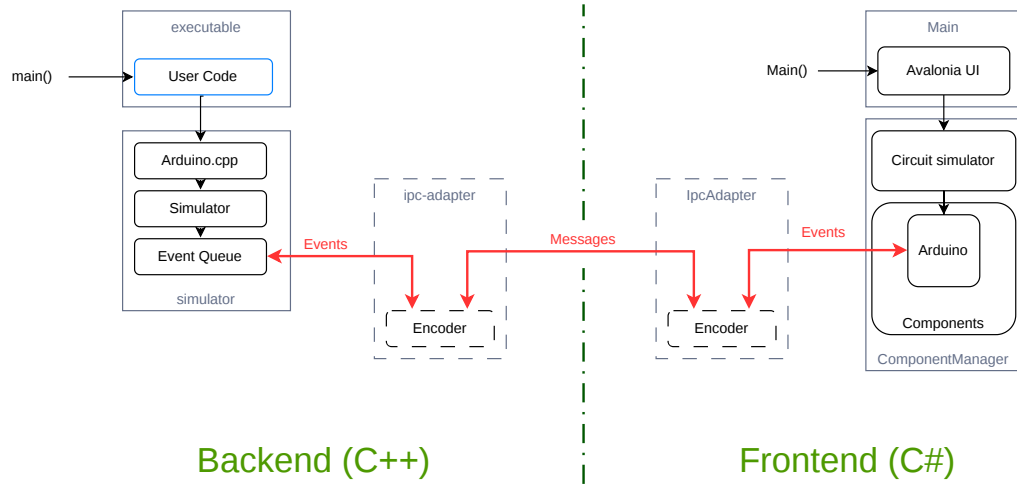


Figure 1.1 Top-level architecture overview

The Figure 1.1 shows the most important systems in each process. The user coded is compiled together with the **executable** module of the backend. The declarations in the header **Arduino.h** prepended to the top of the **.ino** is implemented by a mock library **Arduino.cpp**. For each observable action an event is created and passed to the singleton **Simulator** class to compute and put into an event queue. The event is also communicated to the frontend process.

In the frontend the events are consumed by the **Arduino** component, one of several components predefined as part of the **ComponentManager** project. This project is responsible for loading user-defined components and scenes, which connect the components together in a circuit. The circuit is simulated in the same project. The user interface is implemented in the **Main** project using Avalonia UI as a framework. Interacting with the UI may invoke new events in the **Arduino** component. Those are sent back to the backend's event queue in the same manner as they were received.

The IPC Adapter is an interface implemented in each process responsible for encoding and decoding the events into a standardised format and relaying to the other process.

1.2 Inter-process communication

Implementing processes that depend on each other comes with certain challenges. Processes always run concurrently, so a variety of synchronization techniques must be applied to avoid race conditions. Before that we need to determine how will the processes exchange data. Process is a kernel level mechanism, thus approaches to handle the exchange can vary between operating systems. These mechanisms are what is called *Inter-process communication*, or IPC for short.

1.2.1 Considerations

There is a technical and an engineering reason for splitting the work into more than one process.

The user must be able to have full control over the frontend user interface at all times, even while the Arduino code is being debugged. Every common debugger blocks all running threads of a process, when a breakpoint is hit. If the GUI has been part of the same process as the systems running the simulation, it would freeze and become unresponsive every time a breakpoint is hit. The only way to avoid this behavior is to separate the GUI to a different process. We want to support breakpoints in our project, as they are the most useful tool in a debugger's toolkit.

Splitting the application into multiple processes is actually a common practice among larger pieces of software. The program of each process can be written in a different programming language and thus delegated into teams with different competencies. They mark clear separation of concerns and force a solid interface design. Each process is easily swappable for another implementation with the same interface while the other processes are still running. In our case, the GUI process could be replaced by a web client, a CLI client, or an IDE add-on.

There are more than a few ways to handle IPC on every OS. The ones that we implemented are *TCP stream* and *shared memory*. These are among the most common approaches available both on Windows and Linux. In either case, we must clearly define what the shared data represent. Although shared memory has more flexibility than TCP, we stuck to sending discrete text or binary messages of predetermined format over time, to keep the implementation simpler and the two approaches interchangeable.

Another question is what will we encode in our message. At first we encoded the full state of the simulated Arduino in *snapshots* but due to its various shortcomings we later switched to sending discrete *events*. So the simulator in the backend and its interpretation in the frontend both operate in terms of events, but TCP and shared memory modules only understand the notion of sending and receiving messages with very distinct interfaces. That's why we need to design an adapter in both processes and for both IPC methods that sets up the mechanism and encodes or decodes the passing messages into structured event data the simulation logic understands. The IPC adapter then informs the other systems in it's process any time a new event is received.

1.2.2 Snapshotting

In the early iterations, a snapshot of the Arduino state was sent over one message. That included the pinout state, pin modes, timer, etc. The state was periodically requested by the client with a *Read* message without payload. On a change in the frontend, a *Write* message with the state as payload would be sent to the server and the values of input pins would be overwritten atomically to the backend's internal state.

This approach was easy to implement but flawed. The main issue was the frequency of reads. At lower frequencies, the system is prone to temporal aliasing. **Find temporal aliasing definition.** Some events, like very short impulses, can be missed completely. This is not much of an issue in purely combinational circuits. But we want our frontend to support stateful components, e.g. shift registers present on the Funshield, which are controlled by digital writes less than a microsecond apart.

Highering the frequency to such an extent is not viable.

1.2.3 Events

Right now synchronization between processes is maintained by recording every significant event produced by either the backend or the frontend, and sending it to the other process. Each process is equipped with an event queue, that consumes the events in order of the least recent to the most recent. Consuming an event means invoking the appropriate function based on the type of the event with arguments stored in the event's signature.

The Arduino object in the frontend has no logic on its own, so the only consumed events are those produced by the backend. For that reason a simple queue is sufficient, and no further synchronization is necessary. In the backend however, both simulator's and frontend's events are consumed. Sending an event to the frontend and receiving direct consequences of the event back can involve a significant time overhead. In the meantime the simulator produces more events that are consumed almost instantaneously. We need to ensure that the events are consumed in a valid order. We also need a robust queue implementation that isn't prone to race conditions.

An event is labeled with two timestamps. Timestamps don't have to be based on a real clock, the only requirement is that it is a non-decreasing sequence of numbers. The first timestamp sent is the simulation time, the time as perceived by the Arduino being simulated. That is time given by a clock that is started right before the `setup()` is invoked. It can be manipulated by the user by pausing or slowing down. This is to aid the user while analyzing the logs of the simulator.

The other timestamp included is *Local virtual time* (LVT)¹. Each process holds its own integer value of LVT. This number is increased with newly produced events and events received from the other process. When a decision has to be made on which event to consume next (like in the case of the backend's queue), the event with lower LVT is chosen. The relation $LVT_1 > LVT_2$ means that the event with LVT_1 happens after the event with LVT_2 . In other words, event 1 is the consequence of the event 2 regardless of the real time passed between the events.

The rest of the information stored in an event is its type and arguments. The type corresponds to the action taken after the event is consumed, like writing to a pin or changing pin mode. Each type can define up to 3 integer parameters, e.g for a *write* event that is id of the pin, and a digital value we want to write to it. By limiting the number of arguments, we intend to avoid heap allocation of these events, which could considerably slow down the simulation with programs that produce a lot of events in a short time.

There are many ways to encode such an event into a message. Currently the only encoder implemented in this work is the text encoder, which parses values of events into human readable strings separated by a colon. The messages themselves are separated by a semicolon. This particular encoding is quite space ineffective, but it helps a lot during debugging of the IPC systems.

1.2.4 TCP stream

Messages can be sent and received through a stream of data on a TCP connection. TCP is commonly used to send data over a network, meaning backend and frontend processes can run on different machines. But listening to a connection on the same machine on the same port works just as well. TCP messages are guaranteed to arrive in the same order as they were sent.

The backend implements the TCP server, while the frontend implements the client. The implementations and design considerations behind these are very different based on the language they were written. But in both cases TCP handlers need to run on their own threads, due to how heavyweight they are. There is a buffer to which read bytes are stored from an OS network stream. These are passed as a chunk to a higher level message handler that splits the chunk into individual messages according to predetermined delimiter character. When a message needs to be sent from another thread, it is passed to the message handler where it is posted to the end of a queue and is scheduled to be processed by the OS.

There is a minor limitation of IPC over TCP that makes the debugging experience of the user's code uncomfortable. As we established, hitting a breakpoint will block all running threads of the process, including the thread with the TCP handler. When stepping through the lines of the code where events are produced, the TCP thread has to schedule posting of the message with the event, and it takes time for it to be eventually sent. In many cases the TCP thread won't be fast enough to send the message in the time the main thread progresses to the next line where a breakpoint will be hit again. As a result, consequences of the event won't be reflected in the frontend process until another one or more lines have been stepped through. This is the main reason we decided to also implement IPC with memory mapping, even though TCP is fully functional.

¹The terminology and principles are inspired by the *Optimistic synchronization* paradigm in distributed systems. In this paradigm, the events are consumed as fast as possible, and then rolled back once an event with a lower LVT than the ones consumed appears (so called *Time Warp* mechanism). But Time Warp is not viable for simulating code of a high level language like C++ without custom memory management, and in our implementation it isn't even necessary.

1.2.5 Shared memory

Memory mapping is the functionality of an operating system to relate a region of main memory 1:1 to a region in the address space of a process. This is an important idea in establishing a virtual memory of a process, but OS objects can be mapped as well. It could be either a *memory-mapped file*, contents of a disk file moved to the RAM. Or it could be a special OS object called *shared memory*, living in the kernel heap, specifically designed for IPC. In both cases, multiple processes can map the same memory region to their own address space. Thus they have access to the same data.

The terminology and API differ significantly between Linux and Windows, but with the use of the right libraries, the differences can be abstracted away and used as a single concept. So from now on we refer to the idea of mapping the same memory region to multiple address spaces as *shared memory*.

This tool is very powerful, we can address the shared data by pointers as we would with any other data of the process. There is also no need for system calls once the the memory has been mapped. No special instructions need to be used for reading or writing and all changes are immediately visible on all of the processes that map the same region. As long as we do the writes in the simulating thread, we avoid the problem with delayed observations during debugging we encountered with TCP.

In the shared memory region, we establish a data structure that stores messages in a sequential order. We opted for a circular buffer to fully utilize the limited space of the region. There are actually two non-overlapping buffers in the same region together with a few bytes of metadata about their current state. A process consumes messages from one buffer while producing new messages to another. This is to avoid numerous race conditions that would occur while writing and reading from the same buffer while keep the synchronization mechanism lock-free.

As for the metadata, for each buffer a position after the last consumed and produced message is stored. It is important to never store references as pointers in the shared memory itself. The memory region will be most likely mapped to different addresses in each process. Because of that the *producer offset* and *consumer offset* are regular integers denoting the distance from the first adress of the buffer in bytes. When the offsets are equal, it means the buffer is empty. We don't care about concurrent updates of these offsets, one process changes only one of the offsets for the duration of the simulation.

1.3 Backend

1.3.1 The simulation

The language choice for backend was obvious, the `.ino` file is a syntactically valid C++ code. We include the code in the project by passing it as an argument to the `preprocess.sh` script (`preprocess.bat` on Windows). Inside we use the `arduino-cli` tool with the right arguments to apply very light prerprocessing to the file, like adding function declarations to the top or including the `Arduino.h` header. Important additions are the `#line` directives, which link to the definitions in the original `.ino` file. Thanks to these, breakpoints can be added to the `.ino`

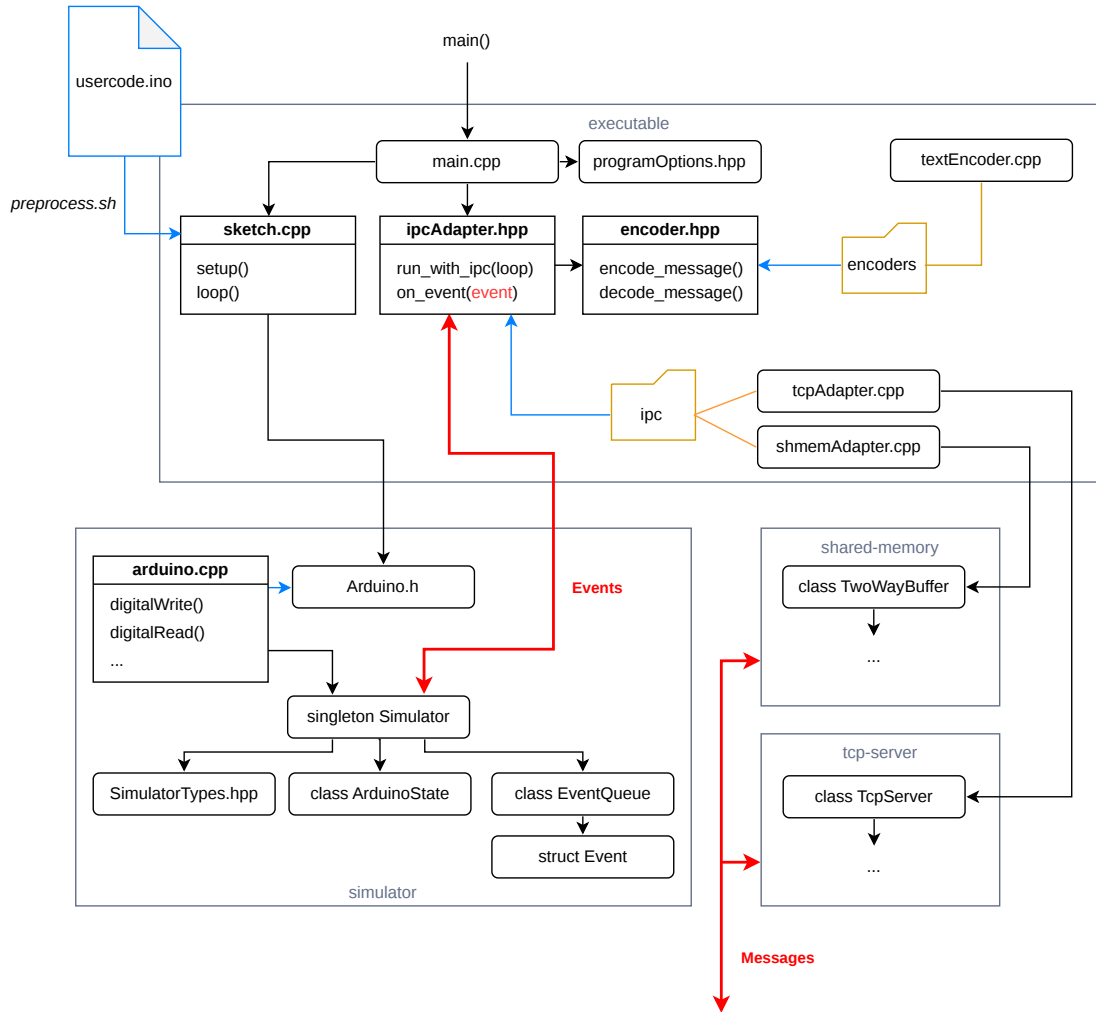


Figure 1.2 Backend architecture

file and debugged as if the file itself was part of the project, completely with stepping through lines and inspection of variables. The file is then saved as `sketch.cpp` in the directory of the `executable` module where it is picked up by the CMake configuration and included in the `main.cpp`. The functions declared by the `Arduino.h` header file, like `digitalWrite` or `millis` are implemented in a custom mock library `Arduino.cpp` in the `simulator` module. Instead of instructing the AVR chip, as the real implementation would, the mock definitions create events and pass them to the `Simulator` for enqueuing and consumption.

The `Simulator` singleton class is the beating heart of the backend. It is responsible for handling the events and the event queue, managing the Arduino state, logging changes and keeping track of time. It is initialized in `main.cpp` and used frequently throughout the `executable` and `simulator` modules. The `Simulator` holds the instance of the `ArduinoState` class, representing the digital states on its pins, current pin modes and the inner timer of the Arduino. It also holds the instance of the `EventQueue`, which stores both local and remote events waiting for consumption. The `SimulatorTypes.hpp` is a header file with very few type definitions commonly used in the `simulator` module.

1.3.2 Events and IPC

The `Event` struct is just a "recipe" for what changes should the `Simulator` do to the `ArduinoState`. It holds a lambda function, corresponding to an event's type that is executed only after the event is put in the event queue and consumed in the order described in the section 1.2.3. The struct defines static functions that create new event instances with these predefined lambdas with the event type's parameters. When a local event is consumed, i.e. the lambda is executed, the event is signalled back to the `executable` module through a callback, that has been passed to the `Simulator` during initialization. This way the event can be sent to an IPC adapter, regardless of the used implementation, without a circular dependency on the `executable` module.

The IPC adapter itself is an interface of two functions declared in the `ipcAdapter.hpp`. `run_simulator_with_ipc` initializes the IPC mechanism. It takes the simulator loop function as an argument and executes it after IPC has been set up. That is because the IPC usually needs to create new threads that cannot be joined before the loop stops running. The loop function itself is defined in `main.cpp` and it contains both the `setup()` and `loop()` calls from the user's code.

The `on_event` function of the adapter is the callback passed to the `Simulator`. It encodes the event into a message and sends over IPC to the remote process. Listening to events of the remote process occurs on a different thread than the one with the simulator loop. Incoming messages are decoded into `Event` structs and sent directly to the event queue in the `Simulator`. `encode_message` and `decode_message` are functions declared in yet another interface in the `encoder.hpp` header. It's implementations are in the `encoders` directory, i.e. only `textEncoder.cpp` at the moment.

Implementations of the IPC adapter are located in the `ipc` directory. The `tcpAdapter.cpp` depends on the `tcp-server` module, while the `shmemAdapter.cpp` depends on the `shared-memory` module. Which one is used depends on the chosen compilation target. The `tcp-server` module contains definition of the TCP server and mechanisms for accepting new connections and handling the stream of data using the `boost::asio` library. The `shared-memory` module contains memory mapping mechanisms using the `boost::interprocess`, implementation of the circular buffer and the producer-consumer pattern using two buffers.

1.3.3 Utilities

There are additional utility modules not shown the figure 1.2. One of them is the humble `logger` used extensively throughout the project. The `ILogger` abstract class exposes the `log` function accepting either a raw string or an `IMessage` abstract struct. Structs inheriting from `IMessage` represent unified formatting of a message with a timestamp and a level of verbosity. The verbosity levels from the highest to lowest are *Debug*, *Info*, *Warning* and *Error*. The messages of higher verbosity levels can be omitted from the logs by setting the appropriate command line option. `ILogger` offers shorthand functions for logging these general messages, but more specific ones can be defined as well.

The module comes with `FileLogger` implementation of the `ILogger`. It creates

or opens a file and dumps the logs inside. To avoid race conditions while writing to the files, two of these loggers are used in the project. One is owned and used by the **Simulator** singleton for logging inner workings of the Arduino. The other is owned by the IPC adapter and used for logging the exchanged messages or potential failures of the IPC mechanisms.

The other utility model is **Timer**. It is used to track time from the start of the simulation. The simulator holds two of these timers, one for *real time* and one for *simulation time*. Although there is no difference between them right now, the intention is to apply effects like stopping or slowing down to the simulation time timer from the frontend, so that functions like `millis` work consistently with disturbances caused by the frontend. Functions returning real time and simulation time are passed to the file logger as a source of its timestamps.

1.4 Frontend

2 Implementation

2.1 Backend

2.1.1 Simulator module

2.1.2 Executable module

2.1.3 IPC Adapters

TCP Adapter module

Shared Memory Adapter module

2.2 Frontend

2.2.1 Component Management project

2.2.2 Main project with Avalonia UI

2.2.3 IPC Adapter project

TCP Adapter module

Shared Memory Adapter module

3 Developer documentation

3.1 Compiling and running Arduino code

3.2 Defining components

3.3 Defining scenes

3.4 IPC interface

Conclusion

List of Figures

1.1	Top-level architecture overview	7
1.2	Backend architecture	12

List of Tables

List of Abbreviations

A Attachments

A.1 First Attachment